IMT Nord Europe – CERI Systèmes Numériques

# Compte rendu de TP

## Internet des Objets & Réseaux

Conception d'une application IoT de relevé
de température et d'humidité

Yassine EL FERGOUCH
Poste n° 42920

Année 2026

# Table des matières

**Introduction** **2**

**1 TP1 – La couche objets : simulation sous SimulIDE** **2**

1.1 Démarche . . . 2
1.2 LED et bargraphe . . . 2
1.3 Capteur DHT22 avec seuil . . . 3
1.4 LCD I2C + DHT22 . . . 4
1.5 Montage final TempHumiX . . . 4

**2 TP2 – La couche réseau : Wi-Fi et protocole MQTT** **5**

2.1 Paramètres utilisés . . . 5
2.2 Prise en main avec MQTTX . . . 5
2.3 Programmation de l'Arduino UNO R4 . . . 6
2.4 Amélioration de la communication . . . 7
2.5 Analyse des trames avec Wireshark . . . 8
2.6 QoS, messages retenus et jokers . . . 9
2.7 Questions générales . . . 10

**3 TP3 – La couche application : IHM Python** **11**

3.1 Client MQTT en console . . . 11
3.2 Thread MQTT . . . 12
3.3 IHM PySide6 – notice de fonctionnement . . . 13
3.4 Archivage SQLite . . . 13
3.5 Affichage des courbes . . . 14

**Conclusion** **15**

# Introduction

Le projet consiste à construire, étape par étape, une chaîne IoT complète autour d'un capteur de température et d'humidité (boîtier *TempHumiX*). Chaque TP correspond à un niveau de l'architecture vue en cours :

| TP | Niveau de l'architecture | Contenu |
|---|---|---|
| TP1 | Niveau 1 – Objets | Simulation SimulIDE : LED, bargraphe, DHT22, LCD I2C |
| TP2 | Niveau 2 – Réseau | Arduino UNO R4 en Wi-Fi, NTP, MQTT, analyse Wireshark |
| TP3 | Niveau 3 – Application | Client Python, IHM PySide6, base SQLite, courbes |

Le capteur porte l'identifiant `42920` (numéro du poste). Tous les topics MQTT sont construits sur ce numéro, par exemple `/iot/TEMPERATURE_42920`.

# 1 TP1 – La couche objets : simulation sous SimulIDE

**Lien avec le cours :** niveau 1 de l'architecture IdO (capteurs / actionneurs / unité de traitement). Le microcontrôleur (ATmega328 de l'Arduino UNO) lit un capteur (DHT22) et commande des actionneurs (LED, bargraphe, afficheur). On reprend l'exemple du cours « allumage d'une LED » puis on ajoute un composant à la fois.

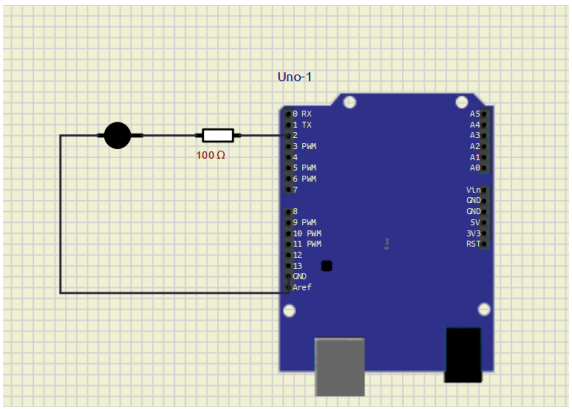
## 1.1 Démarche

On valide chaque brique seule avant de les assembler. Si le montage final ne marche pas, on sait que l'erreur vient de l'assemblage et non d'un composant.

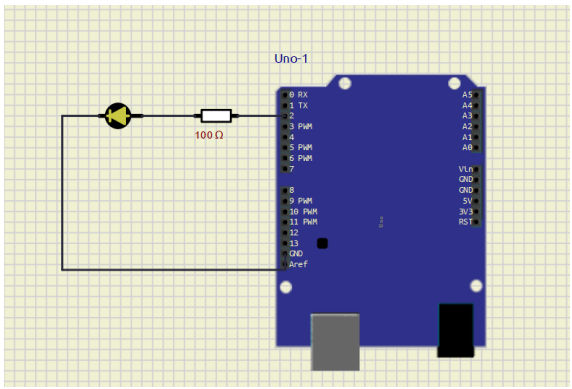
1. LED clignotante (sortie numérique).
2. Bargraphe 10 LED (10 sorties numériques).
3. Capteur DHT22 + seuil de température.
4. Afficheur LCD 16x2 en I2C.
5. Assemblage final : LCD + DHT22 + LED de vie + dateur logiciel.

## 1.2 LED et bargraphe

La LED est reliée à une broche numérique à travers une résistance de limitation de courant (100 Ω). `digitalWrite(HIGH)` met 5 V sur la broche (état 1, LED allumée), `LOW` met 0 V (état 0, LED éteinte).




(a) LED éteinte (état 0)




(b) LED allumée (état 1)

FIGURE 1 – Clignotement d'une LED sur la broche 2

Le bargraphe utilise les broches 3 à 12 : les LED s'allument de gauche à droite. C'est cet actionneur qui sera piloté à distance par MQTT au TP2.

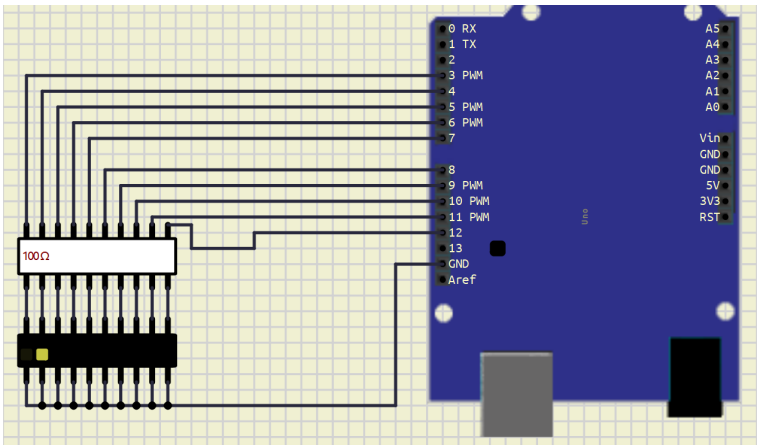
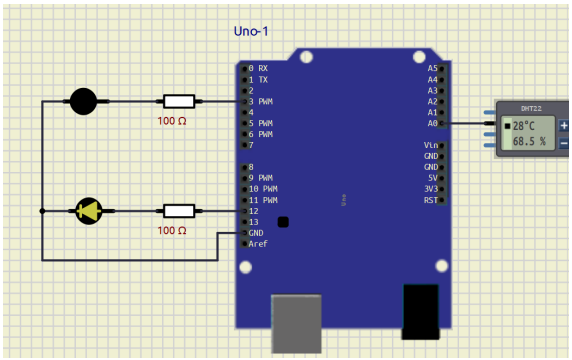


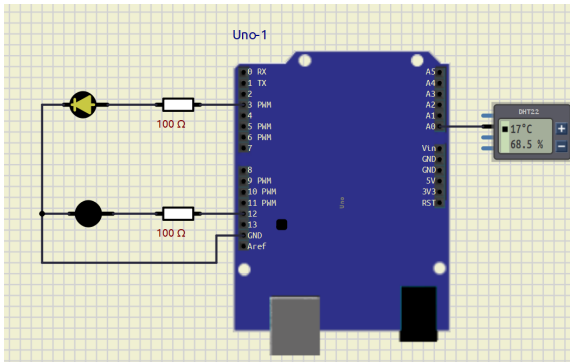

FIGURE 2 – Bargraphe 10 LED (broches 3 à 12)

### 1.3 Capteur DHT22 avec seuil

Le DHT22 envoie température et humidité sur un seul fil (signal numérique). Le programme lit la température toutes les 2 s et la compare à un seuil de 20 °C : LED rouge (broche 12) si $T \geq 20\,°C$, LED verte (broche 3) sinon.




(a) $T = 28\,°C$ : LED rouge




(b) $T = 17\,°C$ : LED verte

FIGURE 3 – Comparaison de la température à un seuil

## 1.4 LCD I2C + DHT22

**Choix des broches :**

— Les LED utilisent des sorties numériques (HIGH/LOW).

— Le DHT22 envoie un signal numérique : il peut se brancher sur `A0`, utilisée ici comme entrée/sortie numérique.

— Le LCD passe par le bus I2C sur `A4` (SDA) / `A5` (SCL) : 2 fils seulement, chaque composant est identifié par son adresse (`0x3E`).

On choisit donc la broche selon le type de signal ou de protocole, pas selon son nom.

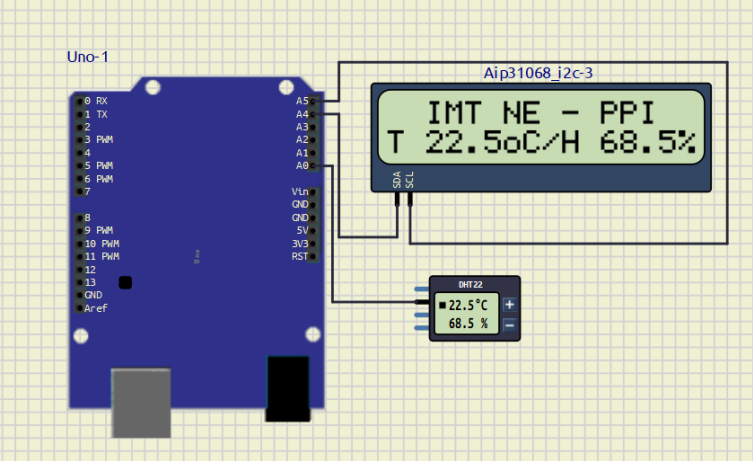



FIGURE 4 – Affichage de la mesure du DHT22 sur le LCD I2C

## 1.5 Montage final TempHumiX

Le programme final tourne sur une boucle cadencée à 100 ms avec un compteur :

— LED de vie : 200 ms allumée sur 500 ms (2 Hz) ;

— toutes les 1 s : mise à jour du dateur logiciel et de la ligne 1 du LCD ;

— toutes les 2 s : lecture du DHT22, ligne 2 du LCD et envoi sur le port série.

Cette « machine d'état » ne bloque jamais la boucle, contrairement à de longs `delay()`.

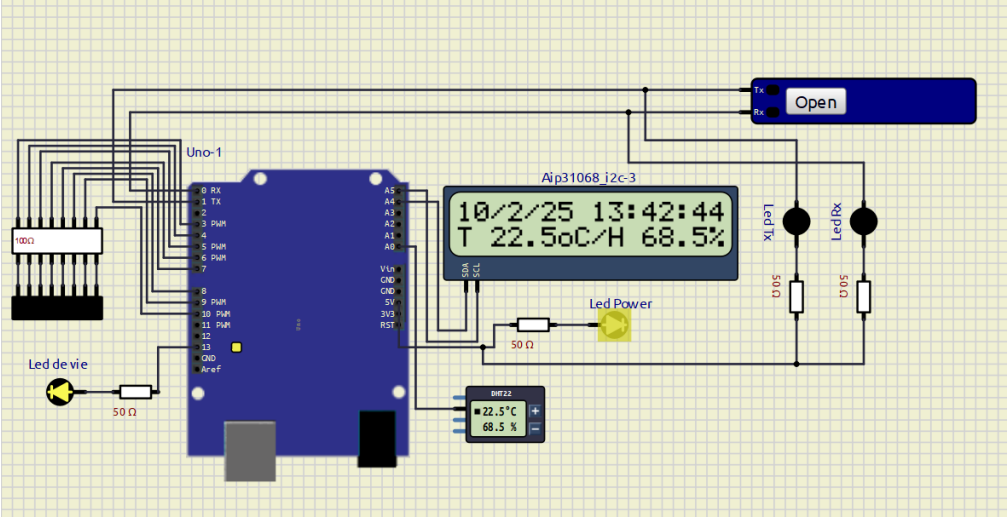



FIGURE 5 – Simulation complète du boîtier TempHumiX

# 2 TP2 – La couche réseau : Wi-Fi et protocole MQTT

**Lien avec le cours :** niveau 2 (réseau / passerelle) et protocoles de la couche application. MQTT est un protocole *publish/subscribe* au-dessus de TCP/IP : les clients ne se parlent jamais directement, tout passe par le *broker* (Mosquitto). C'est le modèle producteur/consommateur du cours : le capteur produit, plusieurs abonnés consomment.

## 2.1 Paramètres utilisés

| | |
|---|---|
| Wi-Fi | SSID `IoT IMT Nord Europe` (2,4 GHz) |
| Broker | `192.168.1.1:1883`, utilisateur `mos_user` |
| Client capteur | `IdO_Client_42920` |
| Publiés | `VERSION_`, `TEMPERATURE_`, `HUMIDITY_`, `GET_DATE_` |
| Souscrits | `COMMAND_`, `BARGRAPH_`, `BLINK_` |

## 2.2 Prise en main avec MQTTX

**Souscrire à la variable d'un autre poste.** On reçoit ses messages sans problème. Le broker ne cloisonne pas les postes : tout client authentifié peut lire le topic d'un autre. C'est pratique en TP mais c'est un problème de confidentialité.

**Souscrire à `TESTS/#`.** `#` est un joker multi-niveaux : on reçoit les messages de *tous* les topics sous `TESTS/`, donc ceux de tous les postes.

**Publier avec *Retain*.** Le message arrive immédiatement à l'abonnement, alors qu'il a été publié avant. Le broker garde le dernier message retenu du topic et l'envoie à chaque nouvel abonné. Sans Retain, un nouvel abonné ne reçoit rien tant que personne ne republie. Pour l'effacer, on publie un message vide avec Retain coché.

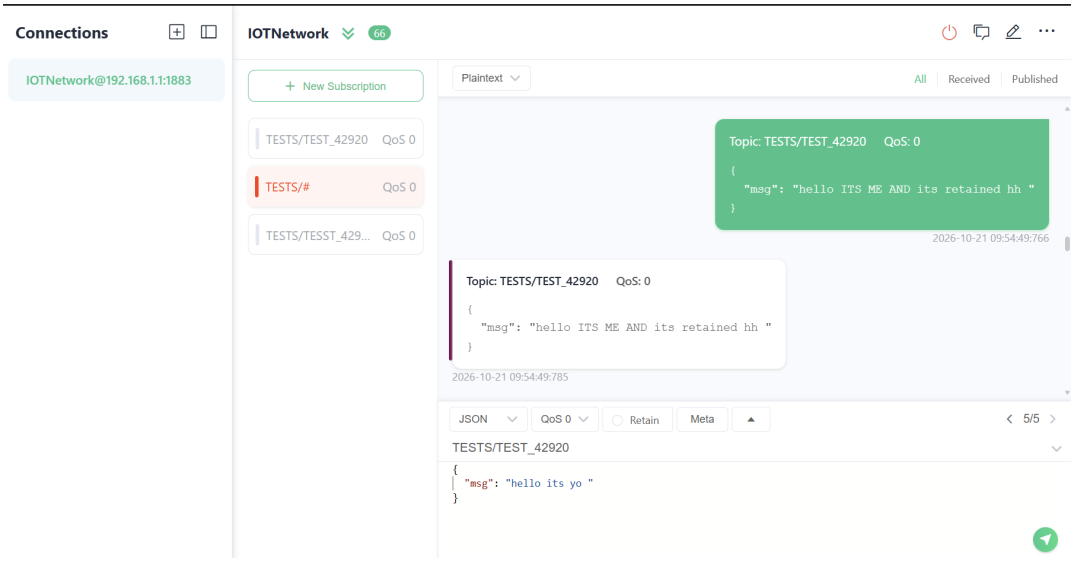



FIGURE 6 – Message retenu reçu dès la souscription (`TESTS/#`)

## 2.3 Programmation de l'Arduino UNO R4

**Passage R3 → R4.** Changement de bibliothèque LCD (`Grove_LCD_RGB_Backlight`), suppression du dateur logiciel (remplacé par le RTC interne), boucle cadencée par `delay()`.

**Wi-Fi.** `setup_wifi()` renvoie `true` si la connexion réussit. Avec le réseau `IoT IMT Nord Europe 5G`, la connexion **échoue** : le module Wi-Fi de l'UNO R4 (ESP32-S3) ne gère que la bande 2,4 GHz. Il ne voit pas du tout les réseaux 5 GHz, donc `setup_wifi()` renvoie `false` (« WiFi echec » sur le LCD).

**NTP + RTC.** Au démarrage, l'heure est lue sur le serveur NTP puis copiée dans le RTC. Le code n'est pas bloquant : en cas d'échec, le LCD affiche « Pas de synchro » et la mesure continue.

**Publication.** `setup_mqtt()` renvoie 0 si OK, sinon le code d'erreur MQTT. Les 4 topics sont publiés toutes les 2 s.

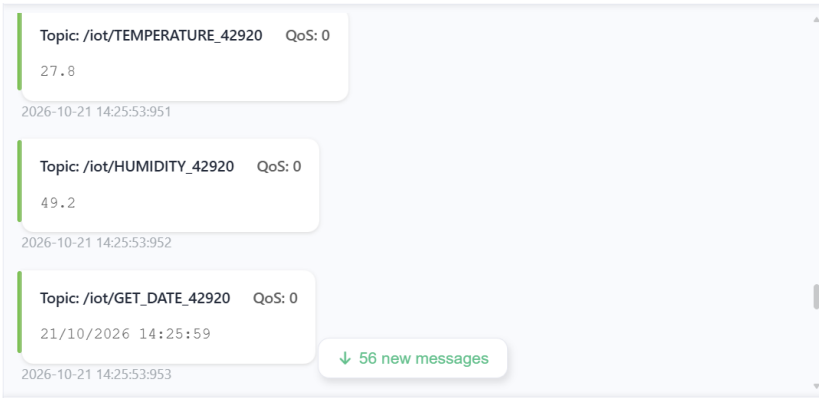



FIGURE 7 – Valeurs publiées par le capteur, vues dans MQTTX

Codes commande : 10 = synchro NTP, 20/30 = démarrage/arrêt mesure, 40/50/60 = clignotement lent/rapide/personnalisé.

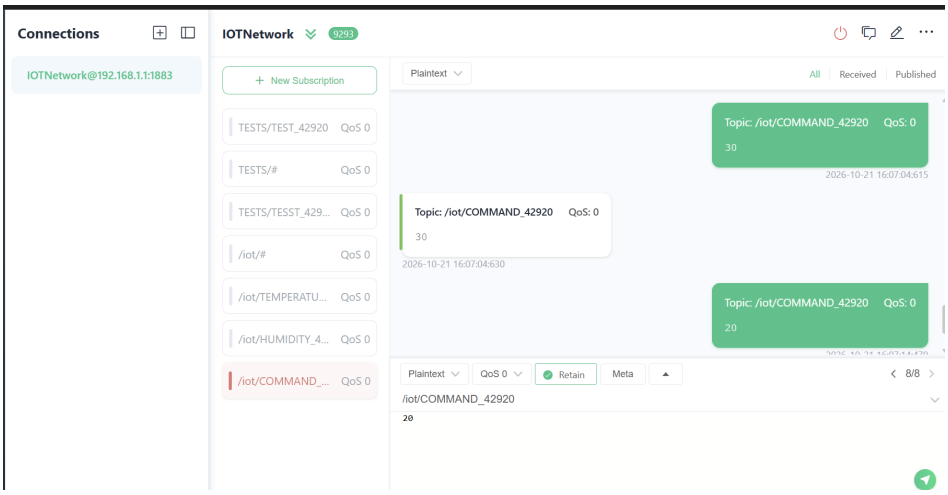



FIGURE 8 – Envoi des commandes 30 puis 20 depuis MQTTX

## 2.4 Amélioration de la communication

Avant : 4 topics toutes les 2 s même sans changement, soit environ 120 messages par minute, la plupart identiques.

a) La version ne change jamais : elle n'est publiée qu'après une synchronisation NTP (commande 10).

b) Température, humidité et date ne sont publiées que si $T$ ou $H$ a changé *au dixième près* (la précision publiée).

```
if ((int)(t*10) != (int)(ancienT*10) || (int)(h*10) != (int)(ancienH*10)) {
  publier_mqtt(); ancienT = t; ancienH = h;
}
```

**Intérêt :** moins de trafic sur le Wi-Fi partagé, moins de travail pour le broker et les abonnés, et moins d'énergie (l'émission radio est ce qui consomme le plus). C'est l'esprit événementiel de MQTT. **Limite :** si la température reste stable, un nouvel abonné ne reçoit rien. On peut corriger cela en publiant avec Retain.

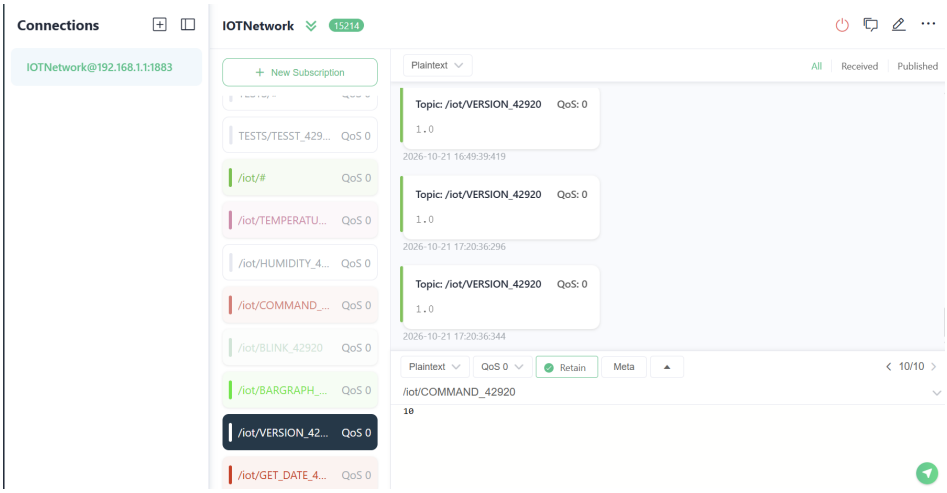



FIGURE 9 – La version n'est publiée qu'à la réception de la commande 10

### 2.5 Analyse des trames avec Wireshark

Capture avec le filtre `tcp port 1883`, puis affichage filtré sur `mqtt`. Le PC (client) est en `192.168.1.168`, le broker en `192.168.1.1`.

| Paquet | Sens | Champs relevés |
|---|---|---|
| CONNECT | client → broker | Client ID `mqttx_258ef9b4`, Keep Alive 60 s, User/Password `mos_user`, MQTT v5.0 |
| CONNACK | broker → client | Return code 0 (connexion acceptée) |
| SUBSCRIBE | client → broker | Topic filter + QoS demandé |
| SUBACK | broker → client | *Granted QoS 0* |
| PUBLISH | deux sens | Topic, message, QoS 0, Retain non, DUP non |
| PINGREQ/RESP | client ↔ broker | 56 octets, en l'absence d'autre trafic |
| DISCONNECT | client → broker | 58 octets, fin volontaire de la session |

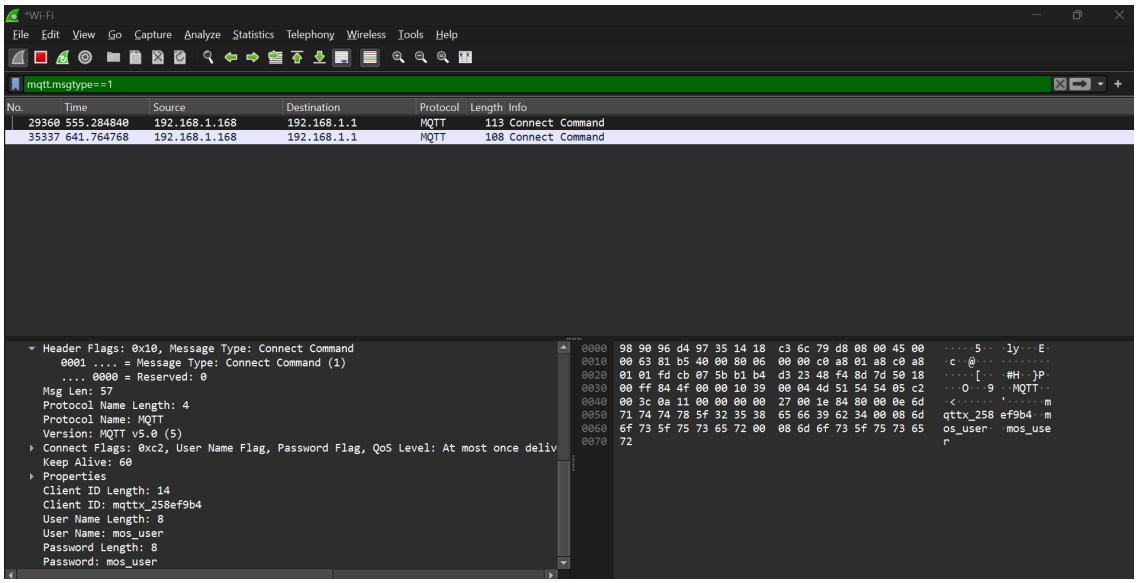



FIGURE 10 – Paquet CONNECT : l'utilisateur et le mot de passe circulent en clair

**Où apparaît le mot de passe ?** Dans le paquet CONNECT, champs *User Name* et *Password*, lisible en clair jusque dans la vue hexadécimale. Le port 1883 n'est **pas chiffré** : toute personne sur le même Wi-Fi peut récupérer les identifiants. Il faudrait passer en MQTT sur TLS (port 8883).

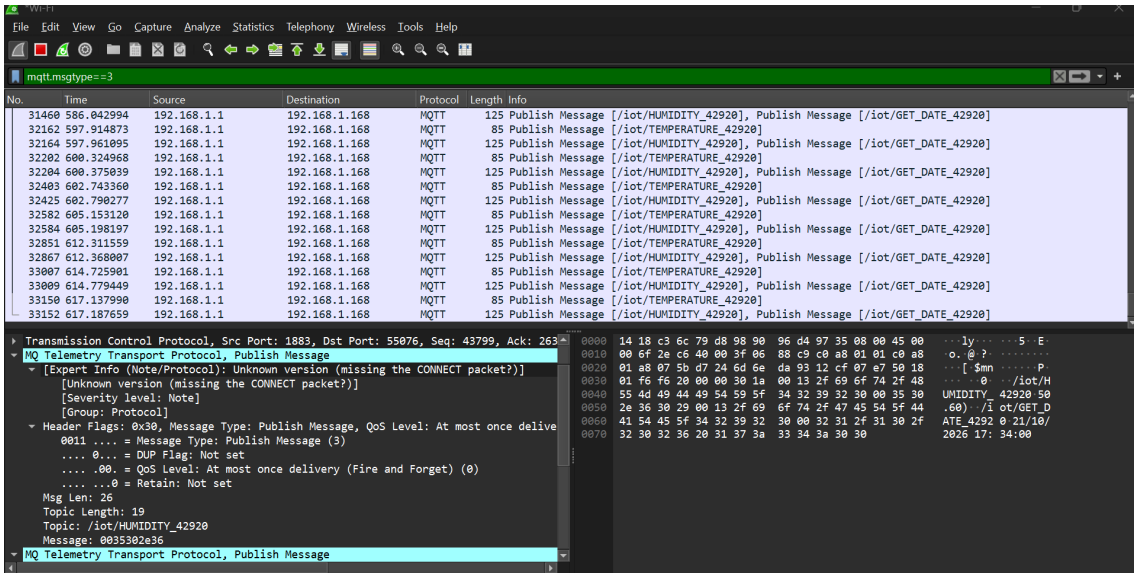



FIGURE 11 – Paquets PUBLISH du capteur (85 et 125 octets)

**Taille d'un PUBLISH de température.** 85 octets sur le réseau, dont seulement une trentaine d'octets MQTT (en-tête fixe de 2 octets, topic `/iot/TEMPERATURE_42920`, valeur `27.8`) ; le reste est l'en-tête Ethernet/IP/TCP. Une requête HTTP équivalente (`POST` + en-têtes `Host`, `Content-Type`, `Content-Length`...) dépasse facilement 200 à 300 octets, sans compter l'ouverture d'une connexion TCP à chaque requête. MQTT garde une seule connexion ouverte et est environ 10 fois plus léger.

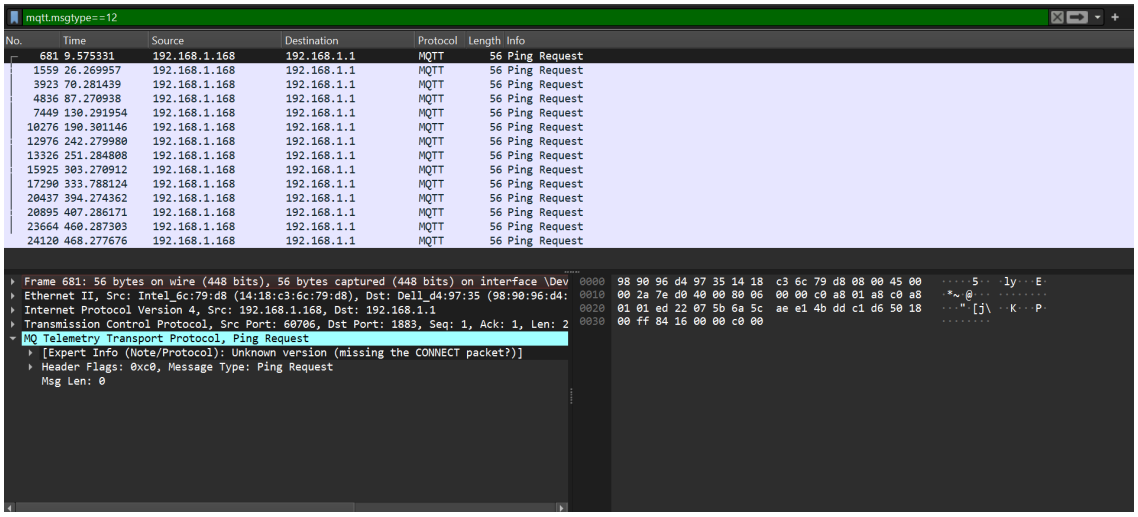



FIGURE 12 – PINGREQ envoyés par le client pour maintenir la session

**Keep Alive.** C'est la durée maximale (60 s ici) sans échange entre client et broker. Si rien n'est envoyé pendant cette durée, le client émet un PINGREQ et le broker répond PINGRESP. Si le broker ne reçoit rien pendant 1,5 fois le Keep Alive, il considère le client mort, ferme la connexion et publie son éventuel *Last Will*.

## 2.6 QoS, messages retenus et jokers

— **QoS 0** : un seul PUBLISH, pas d'acquittement.

— **QoS 1** : PUBLISH + PUBACK (doublons possibles).

— **QoS 2** : 4 paquets PUBLISH, PUBREC, PUBREL, PUBCOMP avec le même identifiant de

message : livraison exactement une fois.

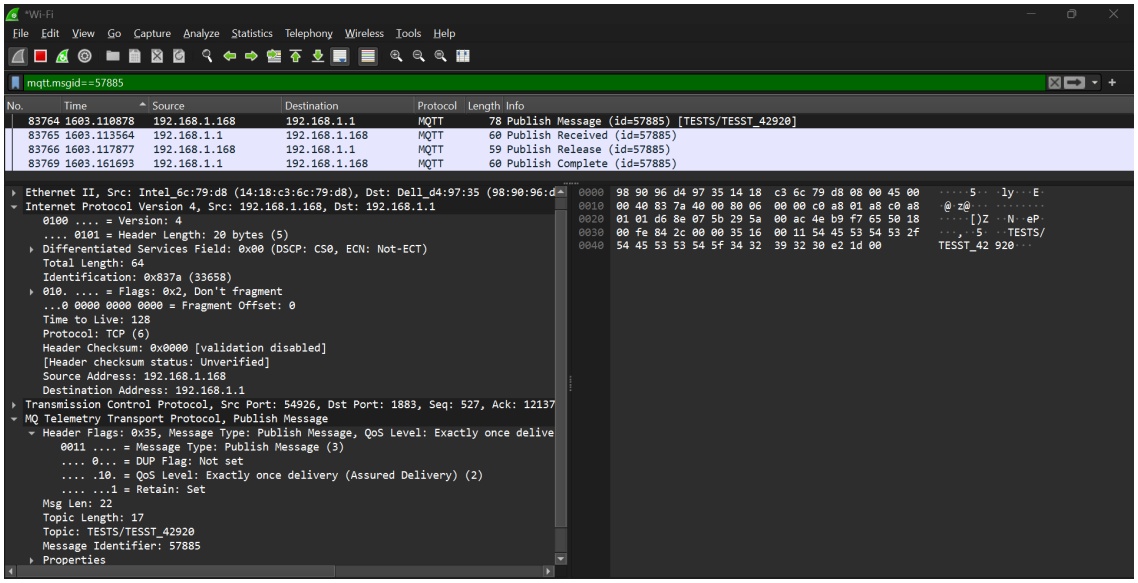



FIGURE 13 – Échange QoS 2 (id 57885) : PUBLISH, PUBREC, PUBREL, PUBCOMP

**Jokers :** `TESTS/+` = un seul niveau (`TESTS/TEST_42920` mais pas `TESTS/a/b`) ; `TESTS/#` = tous les niveaux ; `$SYS/#` = topics internes du broker (clients connectés, messages reçus…), selon les droits donnés par l'ACL.

## 2.7 Questions générales

**1. Acheminement de l'information (modèle OSI / TCP-IP).**

| Couche | Dans notre réseau |
|---|---|
| Application (7) | MQTT (publish/subscribe), NTP pour l'heure |
| Transport (4) | TCP port 1883 (MQTT), UDP port 123 (NTP) |
| Réseau (3) | IPv4 `192.168.1.0/24` : capteur et PC en DHCP, broker en `192.168.1.1` |
| Liaison (2) | Wi-Fi 802.11 côté capteur, Ethernet côté broker ; adresses MAC |
| Physique (1) | Ondes radio 2,4 GHz, câble cuivre |

Le capteur encapsule sa mesure dans MQTT → TCP → IP → trame Wi-Fi. Le RUTX50 (point d'accès) reçoit la trame radio et la renvoie en Ethernet vers le broker : dans Wireshark, l'adresse MAC de `192.168.1.1` est celle d'une machine Dell, le broker tourne donc sur un serveur distinct du routeur. Le broker désencapsule jusqu'à MQTT puis republie vers chaque abonné par le chemin inverse.

**2. Services applicatifs du Teltonika RUTX50.** Serveur DHCP, DNS, NTP, pare-feu, VPN (OpenVPN, IPsec, WireGuard), interface Web/SSH, SNMP, broker et client MQTT, Modbus TCP, envoi de données vers un serveur, gestion à distance (RMS). Dans l'architecture IoT, ces services se situent au niveau **Edge/Fog** : traitement près des capteurs avant le Cloud. Le routeur peut jouer plusieurs rôles :

— **pont** (couche 2) entre le Wi-Fi et l'Ethernet du même réseau local ;

— **routeur** (couche 3) entre le LAN IoT et le WAN (4G/5G) ;

— **passerelle** (jusqu'à la couche 7) quand il convertit un protocole en un autre, par exemple Modbus vers MQTT.

**3. MCU ou MPU ?** Un **MPU** : processeur ARM Cortex-A7 quatre cœurs (Qualcomm IPQ4018, 717 MHz) avec RAM et flash externes. Il exécute un vrai système d'exploitation, **RutOS**, basé sur Linux OpenWrt. Un MCU ne pourrait pas faire tourner autant de services en parallèle.

**4. Connexions Wi-Fi illimitées ?** Non. Le constructeur annonce environ 150 clients simultanés. Le temps d'antenne est partagé (accès CSMA/CA) et la mémoire, le CPU et la plage DHCP sont limités. Risques : un mot de passe Wi-Fi unique et partagé (connu de tous), et tous les appareils sur le même réseau peuvent écouter le trafic non chiffré (MQTT en clair) ou saturer le point d'accès. La topologie est en **étoile** : chaque objet est relié au point d'accès central.

**5. Mesures de sécurité du firmware.** WPA2/WPA3, pare-feu activé par défaut, changement obligatoire du mot de passe par défaut à la première connexion, accès d'administration en HTTPS/SSH, blocage après plusieurs tentatives de connexion ratées, VPN, VLAN et isolation des clients Wi-Fi, mises à jour du firmware à distance (FOTA via RMS). *(Liste à confronter aux notes de version de la version installée sur le routeur.)*

**6. Commentaire des logs Mosquitto.**

- Mosquitto 2.1.2 démarre et charge `mosquitto.conf` ; les modules bridge, persistance, TLS, TLS-PSK et WebSockets sont disponibles.
- Le service tourne sous l'utilisateur `mosquitto` et non `root` : bonne pratique (moindre privilège).
- « Restored 0 . . . » : aucune donnée persistée à restaurer (premier démarrage ou base vide).
- **Avertissements** : les fichiers `passwd` et `acl` sont lisibles par tout le monde et n'appartiennent pas à `mosquitto`. Les futures versions refuseront de les charger. Correction : `chmod 0700` et `chown mosquitto`.
- Le pattern ACL `TESTS/#` ne contient ni `%c` (client ID) ni `%u` (utilisateur) : la règle est la même pour tous, chaque utilisateur peut lire et écrire les topics des autres. C'est ce qu'on a observé avec MQTTX.
- Le plugin de sécurité gère l'authentification (`basic-auth`) et le contrôle d'accès (`acl-check`).
- Écoute en IPv4 et IPv6 sur le port 1883 : port **non chiffré** ; TLS est disponible mais pas utilisé.

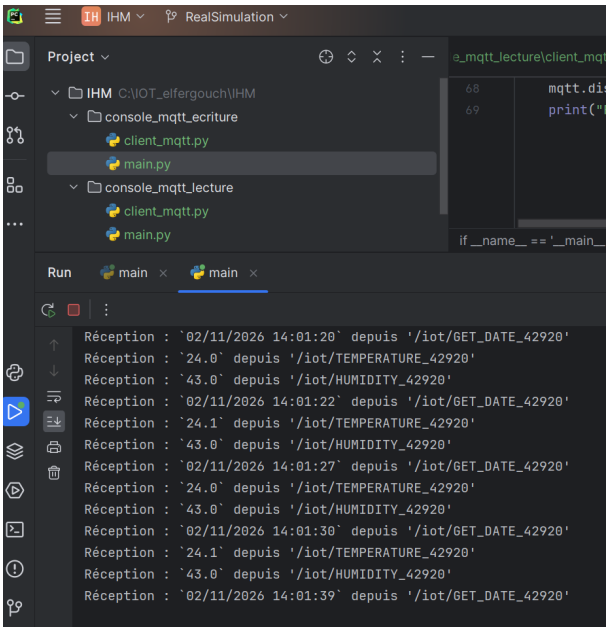
# 3 TP3 – La couche application : IHM Python

**Lien avec le cours :** niveau 3 (application) et éléments de la plateforme IdO du niveau 4 : connectivité (client MQTT), traitement, **stockage** (SQLite) et **visualisation** (courbes). Le PC est un client MQTT comme le capteur : il souscrit aux mesures et publie les commandes.

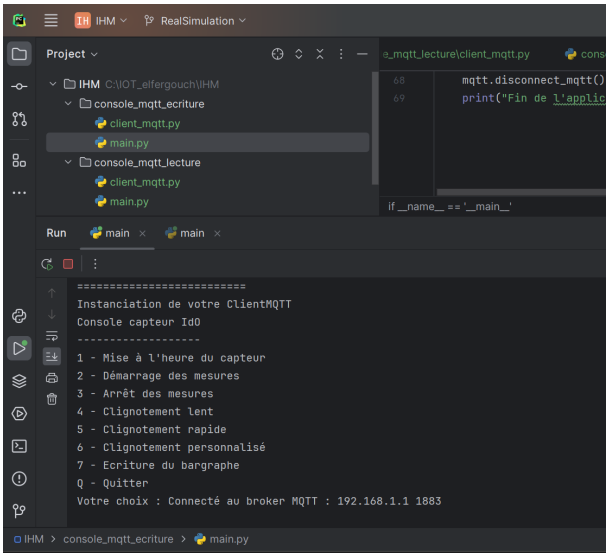
## 3.1 Client MQTT en console

Une classe `ClientMQTT` (bibliothèque `paho-mqtt`) regroupe `connect_mqtt`, `disconnect_mqtt`, `publish_mqtt`, `subscribe_mqtt` et `wait_mqtt`.

- **Lecture** : souscription aux 4 topics du capteur puis `wait_mqtt(0)` (`loop_forever`).
- **Écriture** : menu console, chaque choix publie un code sur `/iot/COMMAND_42920` ou une valeur sur `BARGRAPH/BLINK`. L'arrêt des mesures envoie 30 puis 40 (clignotement lent), le démarrage envoie 20 puis 50 (clignotement rapide) : l'état se voit sur la LED de vie.




(a) Client en lecture




(b) Client en écriture (menu)

FIGURE 14 – Les deux clients console exécutés en parallèle

**Exécution en parallèle.** Deux clients ne peuvent pas avoir le même identifiant : le broker déconnecterait le premier. Le client d'écriture s'appelle donc `Client_E42920`.

## 3.2 Thread MQTT

`loop_forever()` bloque le programme : impossible de lire le menu en même temps. La lecture est donc déplacée dans un `ThreadMQTT` qui appelle `wait_mqtt(timeout=1.0)` en boucle et range les valeurs reçues dans les attributs du client (`temperature`, `humidite`, ...). Le menu reste disponible et le choix 8 affiche les dernières valeurs.

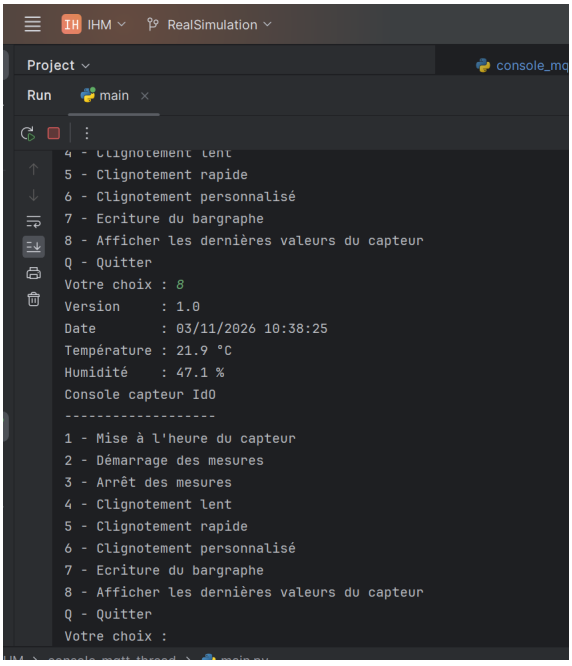



FIGURE 15 – Lecture en arrière-plan + menu de commandes dans un même programme

### 3.3 IHM PySide6 – notice de fonctionnement

L'interface est dessinée avec `pyside6-designer` (`mon_ihm.ui`) puis convertie en Python avec `pyside6-uic`.




FIGURE 16 – Fenêtre principale de l'IHM

1. **Connexion** : vérifier l'adresse IP, le port, l'utilisateur et le mot de passe (pré-remplis, avec contrôle de saisie), puis cliquer sur *Connecter*. Le thread MQTT démarre et un timer rafraîchit l'affichage toutes les 1000 ms.
2. **Affichage** : date/heure du capteur, température et humidité.
3. **Commandes** : synchro NTP (10), arrêter/démarrer les mesures (30/20), clignotement lent/rapide (40/50).
4. **Clignotement personnalisé** : saisir la période en ms (100 à 10000) puis cliquer : la période part sur `BLINK`, puis la commande 60.
5. **Bargraphe** : saisir une valeur de 0 à 1023 puis *Entrée*.
6. **Courbes** : menu *Fichier*, ouvre la fenêtre des courbes.
7. **Déconnexion** : arrête le timer et le thread. Les boutons de commande sont grisés tant qu'on n'est pas connecté.

### 3.4 Archivage SQLite

La base `IOT.db` contient une table `IOT_Data(Id_capteur, Date, Version, Temperature, Humidite)`. La connexion à la base est ouverte *dans* le thread MQTT (SQLite n'accepte pas qu'un objet de connexion soit utilisé depuis un autre thread). Une ligne est insérée à la réception de `GET_DATE`, dernier topic publié par l'Arduino : température et humidité sont alors à jour. La date est convertie au format `AAAA-MM-JJ hh:mm:ss` pour que le tri et les requêtes `BETWEEN` fonctionnent.

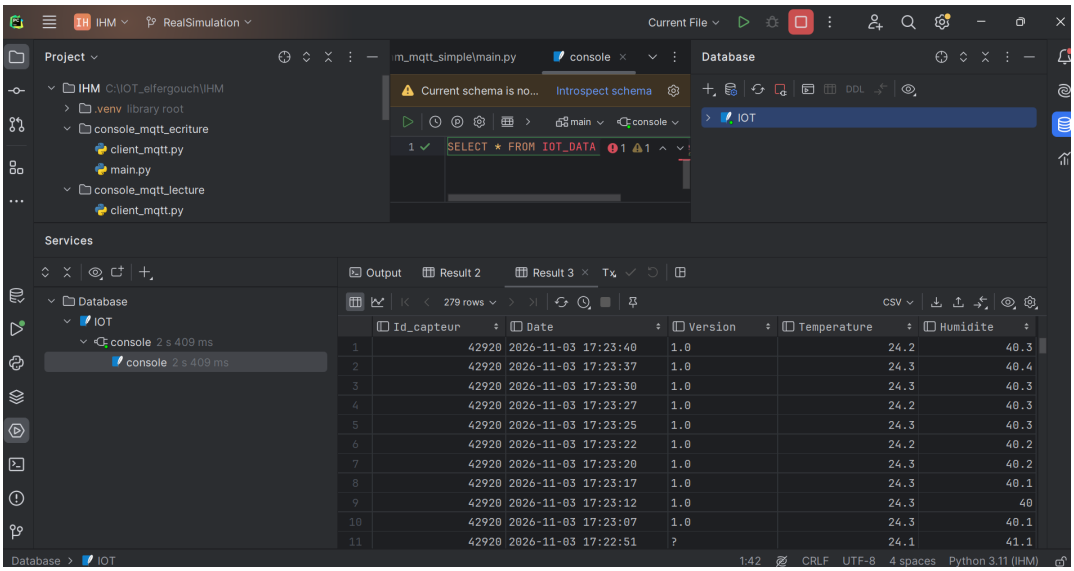



FIGURE 17 – Contenu de la table `IOT_Data` (279 mesures)

## 3.5 Affichage des courbes

La boîte de dialogue `CourbeDialog` lit les mesures entre deux dates (`SELECT ... WHERE Date BETWEEN ? AND ?`) et trace température et humidité avec Matplotlib. La figure est convertie en image et affichée dans un `QGraphicsView`.

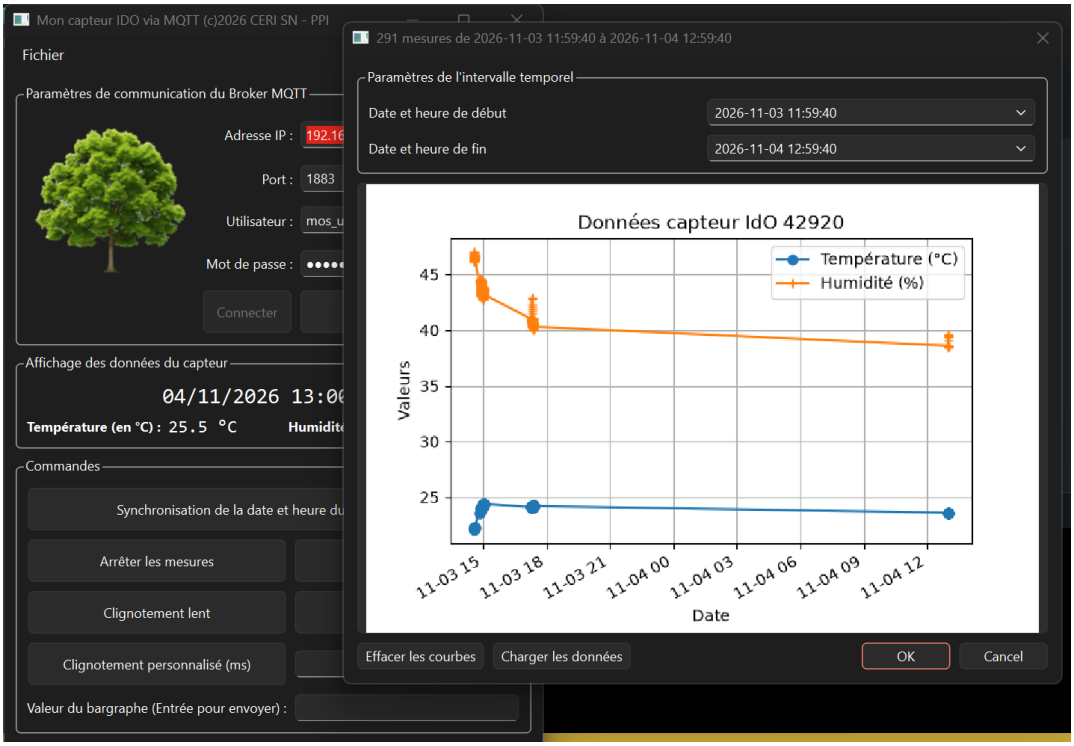



FIGURE 18 – Courbes de température et d'humidité sur 24 h (291 mesures)

# Conclusion

**Acquis.**

— Une chaîne IoT complète, du capteur à la base de données, en suivant les niveaux de l'architecture du cours.

— Programmation embarquée non bloquante (boucle cadencée, RTC, NTP).

— Le modèle publish/subscribe de MQTT, ses QoS, le Retain et les jokers, vérifiés trame par trame dans Wireshark.

— Une IHM Python multi-thread avec archivage et visualisation.

**Difficultés.**

— Le réseau 5 GHz invisible pour l'UNO R4 (bande 2,4 GHz uniquement).

— Le serveur NTP : il a fallu utiliser l'adresse disponible sur le réseau IoT.

— SQLite et les threads : la connexion doit être créée dans le thread qui l'utilise.

— Le format de date : `JJ/MM/AAAA` ne se trie pas, conversion en ISO.

**Perspectives.**

— Sécurité : MQTT sur TLS (8883), un compte et une ACL par capteur (`%u`/`%c`).

— Challenge : sérialiser les mesures dans un seul topic `DATA_` et afficher tous les capteurs de la salle (souscription `/iot/#`).

— Publier les mesures avec Retain pour qu'un nouvel abonné ait tout de suite la dernière valeur.

— Une interface Web pour afficher, commander le capteur et consulter la base à distance.